

## 76. Shadow DOM Slots aur Composition

Kai components (tabs, menus, image galleries) ko render karne ke liye **content** chahiye hota hai. Jaise built-in `<select>` `<option>` items maangta hai, waise `<custom-tabs>` ko tab content aur `<custom-menu>` ko menu items.

html

```
<custom-menu>
  <title>Candy menu</title>
  <item>Lollipop</item>
  <item>Fruit Toast</item>
  <item>Cup Cake</item>
</custom-menu>
```

Component ko ise sahi se render karna hai. Content ko khud copy-rearrange karna possible hai, par shadow DOM me move karne par document ke CSS styles lagna band ho jaate hain, aur code bhi likhna padta hai. Iske bajay Shadow DOM `<slot>` support karta hai jo light DOM ke content se apne aap bhar jaate hain.

### Named slots

js

```
customElements.define('user-card', class extends HTMLElement {
  connectedCallback() {
    this.attachShadow({mode: 'open'});
    this.shadowRoot.innerHTML = `
      <div>Name: <slot name="username"></slot></div>
      <div>Birthday: <slot name="birthday"></slot></div>
    `;
  }
});
```

html

```
<user-card>
  <span slot="username">John Smith</span>
  <span slot="birthday">01.01.2001</span>
</user-card>
```

Shadow DOM me `<slot name="X">` ek "insertion point" hai jahan `slot="X"` wale elements render hote hain. Browser "composition" karta hai: light DOM ke elements ko shadow

DOM ke matching slots me render karta hai.

Result "**flattened DOM**" kehata hai:

```
html

<user-card>
  #shadow-root
    <div>Name :
      <slot name="username">
        <span slot="username">John Smith</span>
      </slot>
    </div>
    ...
</user-card>
```

Flattened DOM sirf **rendering aur event-handling** ke liye hai, ek "virtual" cheez. Nodes asal me **move nahi** hote:

```
js

alert(document.querySelectorAll('user-card span').length); // 2 (abhi bhi use
```

JS document ko "jaisa hai waisa" (flatten se pehle) hi dekhta hai.

**Sirf top-level children** par `slot="..."` chalta hai (shadow host ke seedhe children). Nested elements par ignore hota hai:

```
html

<user-card>
  <span slot="username">John Smith</span>
  <div>
    <span slot="birthday">01.01.2001</span>    <!-- invalid, ignore -->
  </div>
</user-card>
```

Ek hi slot naam ke kai elements ho to wo slot me ek ke baad ek jodte hain.

## Slot fallback content

`<slot>` ke andar kuch likho to wo **default/fallback** content ban jaata hai, jo tab dikhta hai jab light DOM se koi filler na ho:

```
html
```

```
<div>Name: <slot name="username">Anonymous</slot></div>
```

## Default slot: pehla unnamed

Shadow DOM ka **pehla** `<slot>` jiska naam nahi hai "default" slot hai. Light DOM ke saare wo nodes jo kahin aur slotted nahi hain, isme aa jaate hain.

js

```
this.shadowRoot.innerHTML = `  
  <div>Name: <slot name="username"></slot></div>  
  <div>Birthday: <slot name="birthday"></slot></div>  
  <fieldset>  
    <legend>Other information</legend>  
    <slot></slot>  
  </fieldset>  
`;  
;
```

html

```
<user-card>  
  <div>I like to swim.</div>  
  <span slot="username">John Smith</span>  
  <span slot="birthday">01.01.2001</span>  
  <div>...And play volleyball too!</div>  
</user-card>
```

Dono unslotted `<div>` default slot me ek ke baad ek aate hain.

## Menu example

html

```
<custom-menu>  
  <span slot="title">Candy menu</span>  
  <li slot="item">Lollipop</li>  
  <li slot="item">Fruit Toast</li>  
  <li slot="item">Cup Cake</li>  
</custom-menu>
```

Shadow DOM template:

html

```

<template id="tpl">
  <style> /* menu styles */ </style>
  <div class="menu">
    <slot name="title"></slot>
    <ul><slot name="item"></slot></ul>
  </div>
</template>

```

1. `<span slot="title">` `<slot name="title">` me.
2. Kai `<li slot="item">` hain par template me sirf ek `<slot name="item">`, to sab ek ke baad ek us slot me jud jaate hain aur list ban jaati hai.

Valid DOM me `<li>` `<ul>` ka seedha child hona chahiye, par ye flattened DOM hai (rendering ka description), yaha naturally chalta hai.

Click handler (light DOM nodes select nahi kar sakte, to slot par):

```

js

this.shadowRoot.append(tmpl.content.cloneNode(true));
this.shadowRoot.querySelector('slot[name="title"]').onclick = () => {
  this.shadowRoot.querySelector('.menu').classList.toggle('closed');
};

```

## Slots update hona

Bahar ka code menu items dynamically add/remove kare to? **Browser slots ko monitor karta hai aur slotted elements add/remove hone par rendering update kar deta hai.** Light DOM nodes copy nahi hote, sirf render hote hain, to unke andar ke badlav bhi turant dikhte hain. Hume kuch nahi karna. Par agar component ko slot changes ka pata chahiye to `slotchange` event hai.

```

js

this.shadowRoot.firstChild.addEventListener('slotchange',
  e => alert("slotchange: " + e.target.name)
);

```

- Initialization par `slotchange: title` turant chalta hai.
- Naya `<li slot="item">` add karne par `slotchange: item`.
- Slotted element ke **andar ka content** badalne par `slotchange` **nahi** aata (wo slot ka badlav nahi hai). Light DOM ke andruni badlav jaanne ke liye **MutationObserver**.

(`shadowRoot`) par event handlers nahi lagte, isliye uske pehle child par lagaya.)

## Slot API

JS "real" DOM (bina flatten) dekhta hai. Par shadow tree `{mode: 'open'}` ho to:

- `node.assignedSlot`: wo `<slot>` jisme `node` assign hua
- `slot.assignedNodes({flatten: true/false})`: slot me assign hue DOM nodes. `flatten` default `false`; `true` par nested slots (nested components) aur fallback content bhi dikhta hai.
- `slot.assignedElements({flatten: true/false})`: sirf element nodes

Component ko pata rakhna ho ki wo kya dikha raha hai:

```
js
this.shadowRoot.firstElementChild.addEventListener('slotchange', e => {
  let slot = e.target;
  if (slot.name == 'item') {
    this.items = slot.assignedElements().map(elem => elem.textContent);
    alert("Items: " + this.items);
  }
});
```

## Summary

Aam taur par shadow DOM wale element ka light DOM **dikhta nahi**. Slots light DOM ke elements ko shadow DOM ki chuni jagahon par dikhane dete hain.

2 tarah ke slots:

- **Named slots**: `<slot name="X">...</slot>` `slot="X"` wale light children leta hai
- **Default slot**: pehla naam-rahit `<slot>` (baaki unnamed ignore), unslotted light children leta hai
- Kai elements ek slot ke liye ho to ek ke baad ek jodte hain
- `<slot>` ke andar ka content fallback hai

Slotted elements ko slots me render karne ka process **composition**, result **flattened DOM**. Composition nodes ko sach me move nahi karti.

JS access: `slot.assignedNodes/Elements()`, `node.assignedSlot`.

Kya dikha rahe hain ye track karne ke liye:

- `slotchange`: slot pehli baar bharne par, aur slotted element add/remove/replace par (uske children ke badlav par nahi). Slot `event.target` hai.
- Slot content ke andar tak dekhne ke liye **MutationObserver**.

Ab styling: basic rule ye ki shadow elements andar se style hote hain aur light elements bahar se, par kuch khaas exceptions hain (agla chapter).